EXPERIMENTS

12.Write a C program to construct recursive descent parsing for the given grammar

E → TE’

E’ → +TE’ / ∈

T → FT’

T’ → *FT’ / ∈

CODE:

```
#include <stdio.h>
#include <string.h>

char s[100];
int i = 0;

int E();
int T();
int F();

int E() {
   if (T()) {
      while (s[i] == '+') {
         i++;
         if (!T()) return 0;
      }
      return 1;
   }
   return 0;
}

int T() {
   if (F()) {
      while (s[i] == '*') {
         i++;
         if (!F()) return 0;
      }
      return 1;
   }
   return 0;
}
```

```
int F() {
   if (s[i] == 'i') {
      i++;
      return 1;
   }
   return 0;
}

int main() {
   printf("Enter expression (use i for identifier): ");
   scanf("%s", s);

   if (E() && s[i] == '\0')
      printf("Valid Expression");
   else
      printf("Invalid Expression");

   return 0;
}
```

OUTPUT:



13.All languages have Grammar. When people frame a sentence we usually say whether the sentence is framed as per the rules of the Grammar or Not. Similarly use the same ideology, implement either Top Down parsing technique or Bottom Up Parsing technique to check whether the given input string is satisfying the grammar or not.

CODE:

```
#include <stdio.h>

char s[100];
int i = 0;

int E();
int T();
int F();

int E() {
   if (T()) {
      while (s[i] == '+') {
         i++;
         if (!T()) return 0;
      }
      return 1;
   }
   return 0;
}

int T() {
   if (F()) {
      while (s[i] == '*') {
         i++;
         if (!F()) return 0;
      }
      return 1;
   }
   return 0;
}

int F() {
   if (s[i] == 'i') {
      i++;
      return 1;
   }
   return 0;
}
```

```
int main() {
   printf("Enter the string: ");
   scanf("%s", s);

   if (E() && s[i] == '\0')
      printf("String is accepted");
   else
      printf("String is rejected");

   return 0;
}
```

OUTPUT:



14. The main function of the Intermediate code generation is producing three address code statements for a given input expression. The three address codes help in determining the sequence in which operations are actioned by the compiler. The key work of Intermediate code generators is to simplify the process of Code generators. Write a C Program to Generate the Three address code representation for the given input statement.

CODE:

```
#include <stdio.h>
#include <string.h>

int main() {
   char s[50], temp[2];
   int i, t = 1;

   printf("Enter expression: ");
```

```
    scanf("%s", s);

    for (i = 0; s[i] != '\0'; i++) {
        if (s[i] == '*' || s[i] == '/') {
            temp[0] = s[i];
            temp[1] = '\0';
            printf("t%d = %c %s %c\n", t, s[i-1], temp, s[i+1]);
            s[i-1] = '0' + t;
            s[i] = ' ';
            s[i+1] = ' ';
            t++;
        }
    }

    for (i = 0; s[i] != '\0'; i++) {
        if (s[i] == '+' || s[i] == '-') {
            printf("t%d = %c %c %c\n", t, s[i-1], s[i], s[i+1]);
            t++;
        }
    }

    return 0;
}
```

OUTPUT:



15. Write a C program for implementing a Lexical Analyzer to Scan and Count the number of characters, words, and lines in a file

CODE:

```
#include <stdio.h>
```

```
int main() {
    FILE *fp;
    char ch;
    int characters = 0, words = 0, lines = 0, inword = 0;

    fp = fopen("input.txt", "r");

    if (fp == NULL) {
        printf("File not found");
        return 1;
    }

    while ((ch = fgetc(fp)) != EOF) {
        characters++;

        if (ch == '\n')
            lines++;

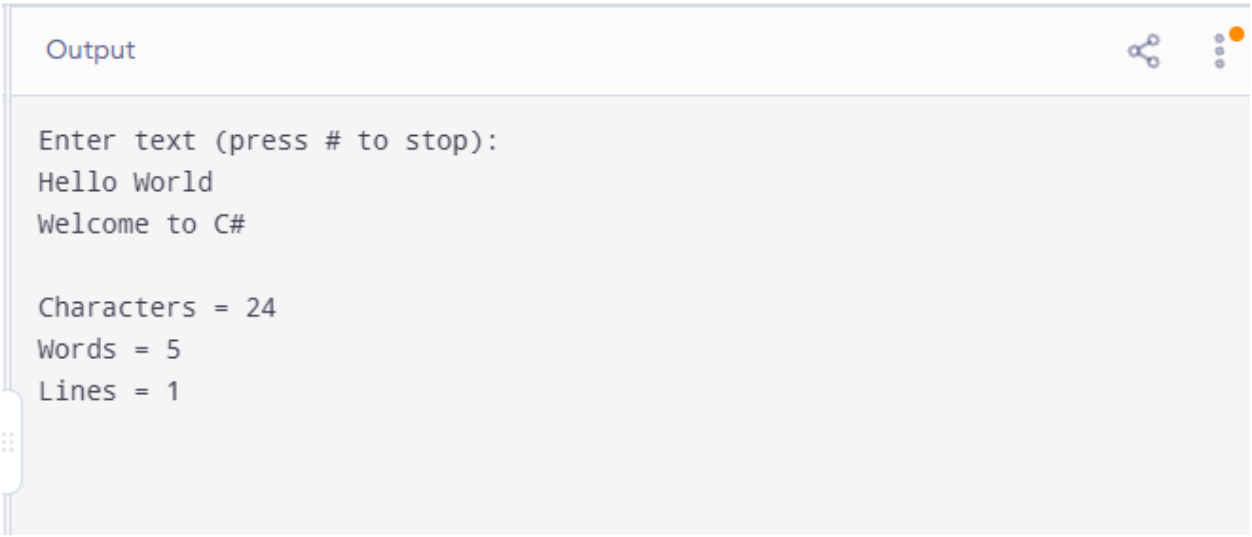
        if (ch == ' ' || ch == '\n' || ch == '\t')
            inword = 0;
        else if (inword == 0) {
            words++;
            inword = 1;
        }
    }

    fclose(fp);

    printf("Characters = %d\n", characters);
    printf("Words = %d\n", words);
    printf("Lines = %d\n", lines);

    return 0;
}
```

OUTPUT:



16. Write a C program to implement the back end of the compiler.
CODE:

```
#include <stdio.h>

int main() {
    char op, a, b, c;

    printf("Enter expression (e.g., a=b+c): ");
    scanf(" %c=%c%c%c", &a, &b, &op, &c);

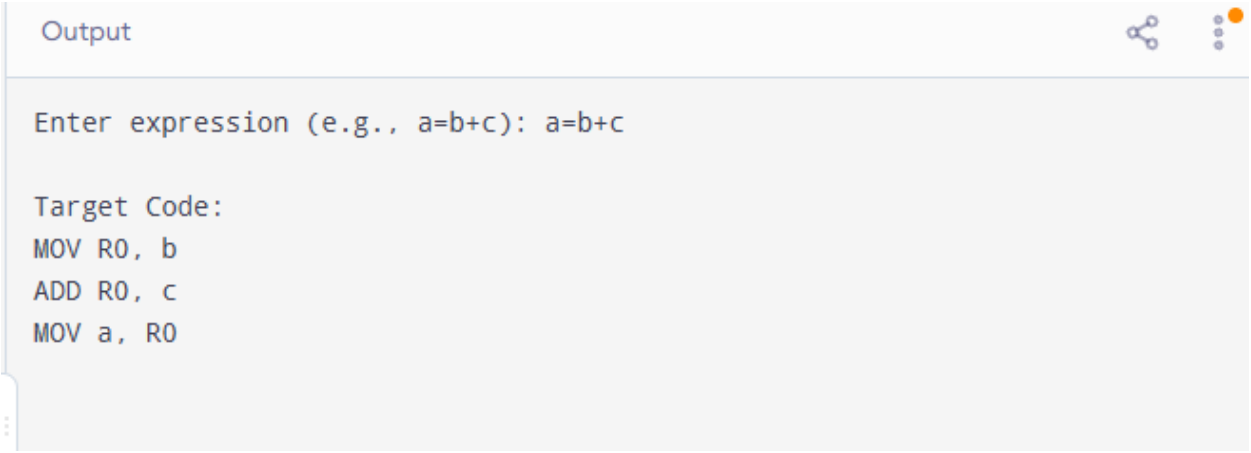
    printf("\nTarget Code:\n");
    printf("MOV R0, %c\n", b);

    if (op == '+')
        printf("ADD R0, %c\n", c);
    else if (op == '-')
        printf("SUB R0, %c\n", c);
    else if (op == '*')
        printf("MUL R0, %c\n", c);
    else if (op == '/')
        printf("DIV R0, %c\n", c);

    printf("MOV %c, R0\n", a);

    return 0;
```

}
OUTPUT:



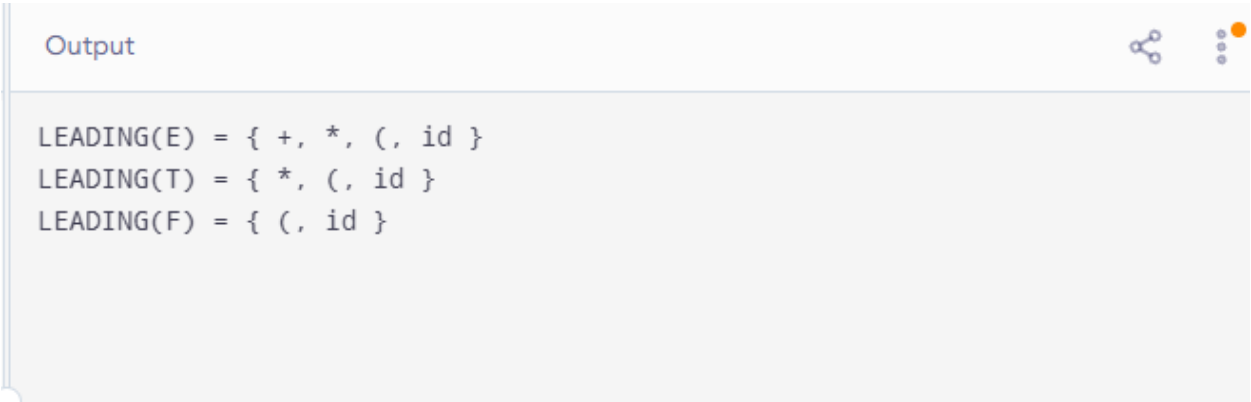
17. Write a C program to compute LEADING( ) – operator precedence parser for the given grammar
E → E + T | T
T → T * F | F
F → (E) | id
CODE:

```
#include <stdio.h>

int main() {
    printf("LEADING(E) = { +, *, (, id }\n");
    printf("LEADING(T) = { *, (, id }\n");
    printf("LEADING(F) = { (, id }\n");

    return 0;
}
```

OUTPUT:



18. Write a C program to compute TRAILING( ) – operator precedence parser for the given grammar

E → E + T | T

T → T * F | F

F → (E) | id

CODE:

```
#include <stdio.h>

int main() {
   printf("TRAILING(E) = { +, *, ), id }\n");
   printf("TRAILING(T) = { *, ), id }\n");
   printf("TRAILING(F) = { ), id }\n");

   return 0;
}
```

OUTPUT:



19. The lexical analyzer should ignore redundant spaces, tabs and new lines. It should also ignore comments. Although the syntax specification states that identifiers can be arbitrarily long, you may restrict the length to some reasonable value. Write a LEX specification file to take input C program from a .c file and count t he number of characters, number of lines & number of words.

Input Source Program: (sample.c)

```
#include &lt;stdio.h&gt;
int main()
{
int number1, number2, sum;
printf(&quot;Enter two integers: &quot;);
scanf(&quot;%d %d&quot;, &amp;amp;number1, &amp;amp;number2);
```

```
sum = number1 + number2;
printf(&amp;quot;%d + %d = %d&amp;quot;, number1, number2, sum);
return 0;
}
```

CODE:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main() {
    char program[] =
        "#include <stdio.h>\n"
        "int main() {\n"
        "int a, b, sum;\n"
        "scanf(\"%d %d\", &a, &b);\n"
        "sum = a + b;\n"
        "printf(\"%d\", sum);\n"
        "return 0;\n"
        "}\n";

    int chars = 0, words = 0, lines = 0, inword = 0;

    for (int i = 0; program[i] != '\0'; i++) {
        char ch = program[i];

        if (ch == '\n')
            lines++;

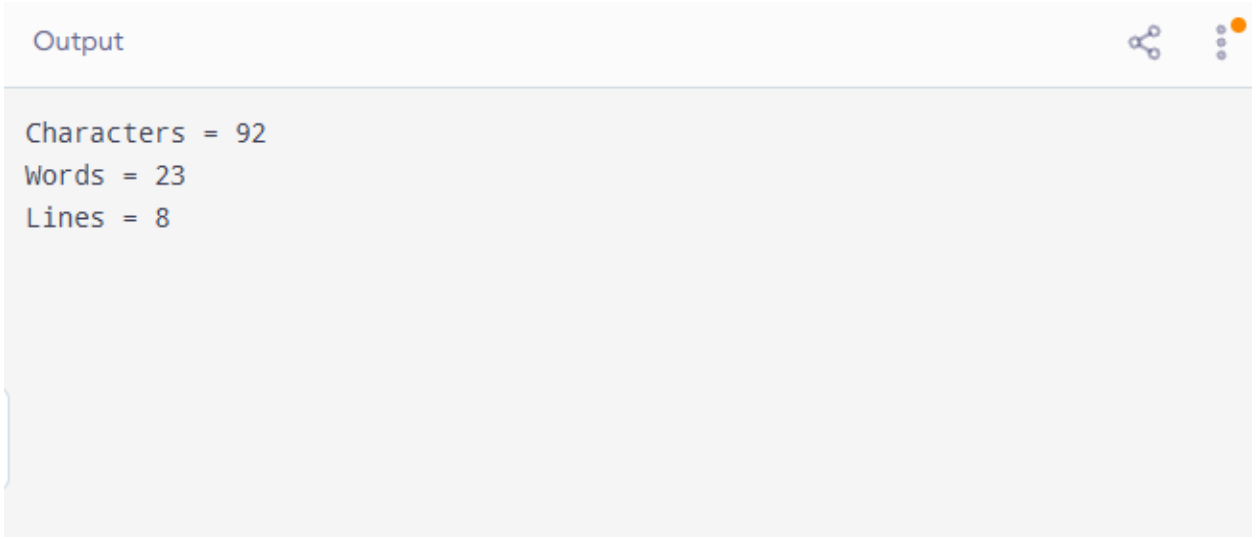
        if (!isspace(ch)) {
            chars++;
            if (!inword)
                words++;
            inword = 1;
        } else {
            inword = 0;
        }
    }

    printf("Characters = %d\n", chars);
```

```
    printf("Words = %d\n", words);
    printf("Lines = %d\n", lines);

    return 0;
}
```

OUTPUT:



20. Write a LEX program to print all the constants in the given C source program file.

Input Source Program: (sample.c)

```
#define PI 3.14
#include&lt;stdio.h&gt;
#include&lt;conio.h&gt;
void main()
{
int a,b,c = 30;
printf(&quot;hello&quot;);
}
```

Write a LEX program to count the number of Macros defined and header files included in the C program.

Input Source Program: (sample.c)

```
#define PI 3.14
#include&lt;stdio.h&gt;
#include&lt;conio.h&gt;
void main()
{
int a,b,c = 30;
printf(&quot;hello&quot;);
```

CODE:

```
#include <stdio.h>
#include <string.h>

int main() {
   char *program[] = {
      "#define PI 3.14",
      "#include <stdio.h>",
      "#include <conio.h>",
      "void main()",
      "{",
      "int a,b,c = 30;",
      "printf(\"hello\");",
      "}"
   };

   int macros = 0, headers = 0, i;

   for (i = 0; i < 8; i++) {
      if (strncmp(program[i], "#define", 7) == 0)
         macros++;
      if (strncmp(program[i], "#include", 8) == 0)
         headers++;
   }

   printf("Number of Macros = %d\n", macros);
   printf("Number of Header Files = %d\n", headers);

   return 0;
}
```

OUTPUT:

